D&A: Woche 9

Sean Findlay

April 20, 2026

1 Shortest Paths

Definition 1.1: Weighted graph

A **weighted graph** $G = (V, E, c)$ is a graph $G = (V, E)$ (where V is the set of vertices (corner points), and E is the set of edges), with a weight function $c : E \to \mathbb{R}$ that assigns each edge in E a weight.
We call $c(e)$ the **weight** (sometimes also the cost) of the edge e.

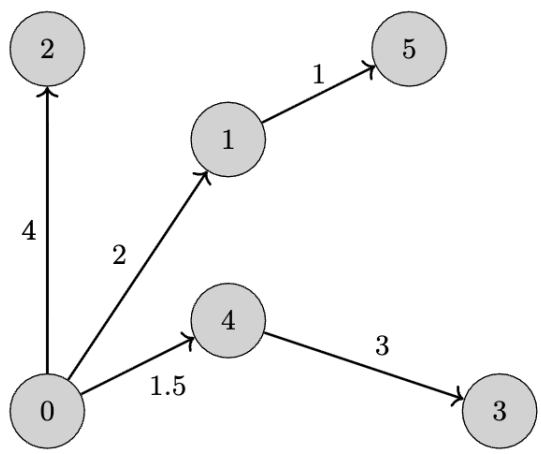

Figure 1: An example of a weighted graph.

Remark 1.2: Weight of a path

Given a weighted graph, we can study the weight of a path. Let us consider a path p from s to t. The weight of p is defined as the sum of the weights of its edges:

$$c(p) := \sum_{i=1}^{k} c((v_i, v_{i+1}))$$

Remark 1.3: Notation for paths

We usually write $p : s \rightsquigarrow t$ or $s \overset{p}{\rightsquigarrow} t$ to indicate that p is a path from s to t, especially when the intermediate nodes do not matter that much.

Definition 1.4: Shortest path

The shortest path in a weighted graph G from node s to node t is defined as the path p with minimum weight $\delta(s,t)$, where

$$\delta(s,t) = \begin{cases} \infty & \text{no path from } s \text{ to } t \\ \min\left\{c(p) : s \overset{p}{\rightsquigarrow} t\right\} & \text{otherwise} \end{cases}$$

Note that the shortest path is not always unique. Our goal will be to simply find one shortest path, no matter which one.

Remark 1.5: Cycles and shortest paths

A shortest path **does not** contain any cycles.

- If the cycle has negative weight (i.e. a cycle such that the total cost of passing through the cycle once is negative), then there exists no shortest path.
- If the cycle has positive weight, then the shortest path will not pass through it.
- If the cycle has weight zero, then we can omit the cycle without changing the weight of the path. So the shortest path (by convention) does not pass through it.

Remark 1.6: Shortest paths obey the principle of optimal substructures

If the shortest path p from s to t goes via a node v, then p must contain the shortest path from s to v and the shortest path from v to t.
Formally, if $p = s \overset{p_1}{\rightsquigarrow} v \overset{p_2}{\rightsquigarrow} t$, then p_1 must be the shortest path from s to v and p_2 must be the shortest path from v to t.
More generally, if $p = (v_1, \ldots, v_k)$ is a shortest path, then all subpaths p_{ij} for $1 \leq i \leq j \leq k$ are shortest paths as well.

Due to this observation, computing the shortest paths from a single source s to a target v essentially requires us to compute **all shortest paths** from a single source to all possible targets in V. We thus now consider this more general problem of **single-source shortest paths**.

2 Dijkstra's Algorithm

Warning: Important assumption

From now on, we assume that all weighted graphs have only **positive** weights.
We also assume WLOG that there are no edges with weight zero. If there was one, then we can just merge it with one of its neighbouring edges.

The problem statement

We consider the weighted graph $G = (V, E, c)$ with positive weights. We want to find all shortest paths from a source node s to all nodes in V.

Initially, we know the shortest path from s to s. It simply has length $\delta(s, s) = 0$. Next, we can look at the outgoing edges from s. The shortest path from s to a neighbouring node v must have a length of at most $c(s, v)$. This provides us with an upper bound.

Now, let us look at the neighbouring node v of s with the smallest upper bound. Could we reach v from s via a path with shorter length than that? No! Because if we could, then v wouldn't be the node with the smallest upper bound. Let us summarise this:

Remark 2.1

Let v be the out-neighbour of s with the minimum edge weight, i.e.

$$c(s, v) \leq c(s, w) \ \forall w \in N^+(s)$$

Then, the shortest path from s to v is $c(s, v)$.

After this, we have found...

- the shortest path from s to another node v
- and we know the upper bounds $d_s[x]$ on$\delta(s, x)$ for all $x \in N^+(s)$.

We can now repeat this process for all outgoing edges of the node v and update the upper bounds of its out-neighbours accordingly. We then have the shortest paths to s and v as well as upper bounds $d_s[\cdot]$ on all nodes in $(N^+(s) \cup N^+(v)) \setminus \{s, v\}$ with $d_s[\cdot] \geq \delta(s, v)$. We note that $d_s[x]$ is the length of the shortest path from s to x that only potentially uses v as an intermediate node, thus $d_s[x]$ is the length of the shorter of (s, x) and (s, v, x).

Let w be the node (different from s and v) with the smallest upper bound $d_s[w]$. We can conclude that $d_s[w] = \delta(s, w)$. Why? If there was a better path, it would have to use the cheapest outgoing path from s. If another path was cheaper, then this node would be our node with the smallest upper bound (and not w). So we ahve found the shortest path to w.

2.1 Formulation of the greedy Dijkstra's algorithm

1. We have a weighted graph $G = (V, E, c)$. Start with $S = \emptyset,\ U = s,\ R = V \setminus s$.

- S is the set with the known shortest path,
- U is the set of nodes reachable from S for which we know an upper bound,
- R is the set for which we do not know an upper bound (other than ∞)

Repeat the following until $S = V$ and hence $U = \emptyset$

2. In each iteration, we add the node u from U with the **smallest** upper bound $d_s[u]$ to S and remove it from U. We set its shortest path length to $d_s[u]$ and update d_s for all nodes in $N^+(u)$.

   More concretely,

   - If there is an edge (u, a) to a node $a \in S$, then this edge is ignored.
   - If there is an edge to a node $b \in U$ with $d_s[b] \leq d_s[u] + c((u, b))$, then this edge is ignored.
   - If there is an edge to a node $c \in U$ with $d_s[c] > d_s[u] + c((u, c))$, then the upper bound of c is updated to $d_s[c] \leftarrow d_s[u] + c((u, c))$.
   - If there is an edge (u, d) to a node $d \in R$ (a node with no known upper bound), then d is added to U and its upper bound is set to $d_s[d] \leftarrow d_s[u] + c((u, d))$.

After i iterations, we have $|S| = i$. Hence, the algorithm terminates after n iterations and claims to have found the shortest path to all nodes in V.

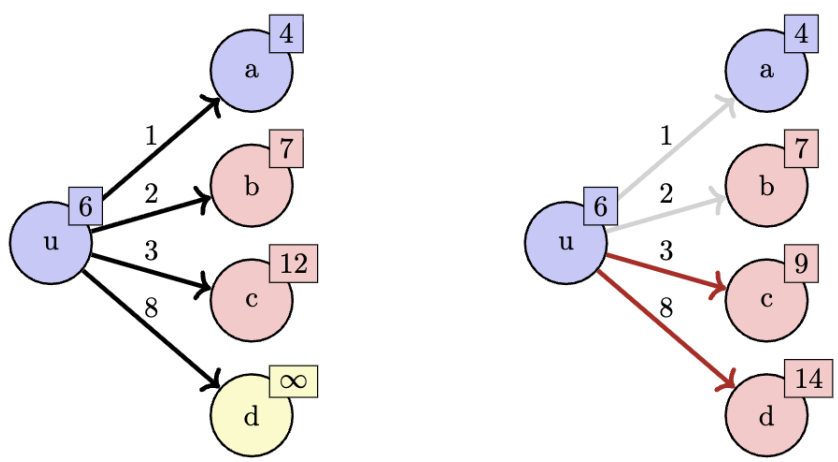

Figure 2: An example of updating the out-neighbours of a node using the steps described above.

Lemma 2.2

Let $v_1, \ldots, v_n$ be the order in which the nodes are added to S. Then,

$$\delta(s, v_i) \leq \delta(s, v_j) \quad \text{for } 1 \leq i \leq j \leq n \text{ and } d_s[v_i] = \delta(s, v_i) \text{ in iteration i.}$$

In simple terms, this lemma proves why Dijkstra's algorithm never has to change its mind. S is the "settled set". These are nodes where the algorithm is sure that we know the shortest paths to them. $v_1, \ldots, v_n$ is the exact order in which we work out the shortest path for each of these nodes. $\delta(s, v_i)$ is the true, objective shortest distance between s and v_i. $d_s[v_i]$ is our current estimated distance to node v_i.

The lemma claims that: if v_i is added to S before v_j, then the true shortest distance from s to v_i is smaller than the true shortest distance from s to v_j.

It also states that at the exact moment when v_i is added to S, its estimate $d_s[v_i]$ is exactly correct and identical to $\delta(s, v_i)$.

```
Input: Weighted Graph G = (V, E, c) with c ≥ 0, starting point s ∈ V
Output: Length d_s of the shortest paths from s and predecessor π_s for each node

foreach u ∈ V do
    d_s[u] ← ∞; π_s[u] ← null
end
d_s[s] ← 0; U ← {s}
while U ≠ ∅ do
    u ← arg min_{u∈U} d_s[u]; U ← U \ {u}
    foreach v ∈ N⁺(u) do
        if d_s[u] + c(u, v) < d_s[v] then
            d_s[v] ← d_s[u] + c(u, v)
            π_s[v] ← u
            U ← U ∪ {v}
        end
    end
end
```

Figure 3: Pseudocode for Dijkstra's algorithm

Data structure for U

We store U in a min heap. This is because we need to be able to insert, as well as extract the minimum.
In order to update priorities, we use **lazy deletion**. When we want to decrease the priority of a node k from p to $p' < p$, we leave the element (p, k) untouched and add an additional element (p', k) to the heap. When a node is extracted from the min heap from the first time, (with its smallest priority), we can mark it as deleted. Then, when we next encounter the node with a larger priority, we can ignore it and move to the next minimum.

```
Input: Weighted Graph G = (V, E, c) with c ≥ 0, starting point s ∈ V
Output: Length d_s of the shortest paths from s and predecessor π_s for each node
foreach u ∈ V do
    d_s[u] ← ∞;  π_s[u] ← null
end
S = {}; d_s[s] ← 0; U ← {s}
while U ≠ ∅ do
    d, u ← ExtractMin(U)
    if u ∉ S then
        S ← S ∪ {u}
        foreach v ∈ N^+(u) do
            if d + c((u, v)) < d_s[v] then
                d_s[v] ← d + c((u, v)); π_s[v] ← u
                Insert(U, (d + c((u, v)), v))
            end
        end
    end
end
```

Figure 4: Pseudocode for Dijkstra's algorithm with lazy deletion

2.2 A* Algorithm: Dijkstra with a heuristic

If we already know where the goal is, then why bother exploring nodes which go in the wrong direction? This is the idea of the A* algorithm, we introduce a heuristic function. A possible choice for the heuristic is the Manhattan distance to the goal. Below is the pseudocode for A*:

Let us now introduce properties of the heuristic function $\hat{h}$.

Definition 2.3: Optimistic heuristic

A heuristic $\hat{h}$ is optimistic, if

$$\hat{h}(v) \leq \delta(v, t)$$

where δ is the true shortest distance.

Input: Positively weighted Graph $G = (V, E, c)$, starting point $s \in V$, end point $t \in V$, estimate $\hat{h}$

Output: Existence and length of a shortest path from s to t

```
foreach u ∈ V do
    d_s[u] ← ∞; f̂[u] ← ∞; π[u] ← null
end
d_s[s] ← 0; f̂[s] ← ĥ(s); U ← {s}; S ← {}
while U ≠ ∅ do
    u ← ExtractMin_f̂(U); S ← S ∪ {u}
    if u = t then return success
    foreach v ∈ N⁺(u) with d_s[v] > d_s[u] + c((u, v)) do
        d_s[v] ← d_s[u] + c((u, v)); f̂[v] ← d_s[v] + ĥ(v); π[v] ← u
        U ← U ∪ {v}; S ← S \ {v}
    end
end
return failure
```

Figure 5: Pseudocode for A*

Definition 2.4: Monotone heuristic

A heuristic $\hat{h}$ is monotone, if

$$\hat{h}(u) \leq c((u, v)) + \hat{h}(v)$$

holds for all $(u, v) \in E$.

Lemma 2.5

If $\hat{h}$ is optimistic, then A* terminates with the correct result, namely the shortest path from s to t.

Conclusion on heuristics

If $\bar{h}$ is **optimistic** and **monotone**, then the A* algorithm is correct and efficient. In practice, if underestimates are easy to compute (e.g. the distance as the crow flies), this can lead to drastic improvement of Dijkstra's algorithm.